

AMMBER — Motivational Modelling Local Editor

Progress Report — Weeks 1–6 (27 Jul – 6 Sep 2026)

Team Deepsick · Client: Leon Sterling · Tutor: Kerr Yuan

Member	GitHub	Role
Jinbao Liu	lkevincc0	Tech Lead / Full-stack Developer (architecture, CI/CD, repo strategy, client liaison)
Zihan Xie	photozity-ally	Frontend Developer (canvas UI, image export/import)
Huimin Peng	ViolaHM-14	Frontend Developer (feedback drawer/panel, node binding)
Zecheng Huang	ZechengHuang-unimelb	QA / Test Lead / Developer (test plan, Vitest suites, bug triage, share feature)
Yuhao Xu	yuhaox5	DevOps & Tooling / Developer (CI pipeline, lint, lockfiles, Docker)

1 Project Overview

AMMBER is a web-based motivational modelling tool used in teaching and research for roughly fifteen years. We are extending the existing React/TypeScript editor (fork: github.com/lkevincc0/mm-local-editor) so that lecturers and reviewers can give *structured, element-level feedback* on motivational models, and so that models can be shared and carried as portable files. A Week 3 client scope change (*no cloud hosting or shared database*) was absorbed within the same week by pivoting to a fully local architecture — with zero sprint disruption.

Delivered in Weeks 1–6 (all via reviewed PRs, continuously deployed to [GitHub Pages](#)):

- **Local project workspace** — create/open/rename/delete projects persisted in-browser, no server (PR [#3](#)).
- **Unified editorial theme & dockable canvas editor** — consistent visual identity; canvas fits and centres the model on load (PRs [#3](#), [#4](#)).
- **Feedback feature end-to-end** — closable draw.io-style side panel, node-bound comments with click-to-locate, authoring, nested replies, editable profile header, generated avatars (PRs [#5](#), [#6](#), [#9](#), [#11](#), [#19](#)).
- **Image-only project import/export** — project JSON (name + feedback) embedded invisibly into exported PNG/SVG, so one image is a full portable backup (PR [#7](#)); compact share links via deflate compression (PR [#20](#)).
- **QR-code project sharing** — modernised share dialog inside the editor (PR [#15](#)).
- **Deployment & robustness** — adaptive router basename, GitHub Pages asset paths, deploy permissions (PRs [#2](#), [#12](#), [#13](#)).
- **Quality pipeline** — 20 Vitest test files plus Cypress E2E, all run by GitHub Actions on every push and PR; four reported bugs triaged and fixed with regression coverage (PRs [#14](#), [#17](#), [#18](#), [#23](#)).

2 Process

2.1 Agile Ceremonies

Two-week sprints after a kick-off Sprint 0; every ceremony produces a written artefact in the team's **Jira project** (37 tickets to date: 18 user stories, 18 tasks, subtasks; sprint backlogs, bug tickets, retrospectives and decision records) and every outcome is traceable to [the repository](#).

- **Weekly team meeting** (every Wednesday, W3–W12, full attendance to date): standing agenda (review merged PRs → sprint check → planning → impediments), minutes with action items, owners and due dates.
- **Sprint planning & retrospective** at every sprint boundary. Retrospective actions are adopted and verified next sprint: after S1, a ~400-line PR size cap, clarified QA-vs-tooling role boundary, and a CI check against stray submodules; after S2, a 24-hour PR review SLA, a rendering regression test, and earlier test-plan execution.
- **Client consultation** at least fortnightly; every request is logged and re-prioritises the backlog (§2.3).
- **Sprints**: S0 (W1–2) kick-off; S1 (W3–4) Git workflow + first vertical slice — completed; S2 (W5–6) feedback end-to-end, image import/export, QR sharing — completed; S3 (W7–8) quality hardening, S4 (W9–10) Docker + docs, S5 (W11–12) release v1.0 — planned.

2.2 Decision Making, Meetings & Internal Communication

Options are listed with rationale, every member is consulted, and the outcome is recorded in a **decision log** (9 decisions to date, tracked in Jira). Key in-period decisions: evolve the existing React/Vite editor rather than rebuild on Next.js (consensus 5/5); GitFlow-lite branching with PR reviews; feature-ownership work allocation; feedback as a closable side panel (team vote 4/5); drop the database for fully local persistence; embed project JSON in exported images (proposed by Tech Lead, feasibility validated, consensus). The Week 3 client constraint triggered an immediate re-evaluation before any sunk cost accrued.

Communication runs through four mutually reinforcing channels: **Slack** for day-to-day coordination; **GitHub PR**

reviews as the archived record of technical discussion; the **Jira project** as a single source of truth (backlog, sprint boards, bug tickets [KAN-21–KAN-23](#), decision log), reconciled against git history weekly; and **pair collaboration** (the frontend pair cross-review each other's PRs; QA and DevOps coordinate on test plan and CI). Issues surface and resolve fast: the stray nested submodule breaking fresh clones was fixed the same week it was raised (PR [#8](#)); the Sprint-2 re-render regression was fixed within days with a regression test added (PR [#6](#)).

2.3 Communication with the Client

Fortnightly check-ins and demos, in motivational-modelling vocabulary (goals, stakeholders, feedback on elements) rather than implementation detail. All client feedback to date has been actioned:

Week	Client feedback	Impact	Status
W3	No cloud hosting / shared database	Backend scope dropped; local workspace promoted to top priority; two roles re-scoped to QA and DevOps	Actioned (S1)
W4	Prioritise feedback workflow over extra export formats	Feedback moved to top of S2 backlog	Actioned (S2)
W6	S2 demo accepted; wants packaged deployment & user docs	Docker packaging + user manual scheduled as S4 high priority	Scheduled (S4)

3 Artefacts

3.1 Requirements

Elicited with the Do/Be/Feel method into a motivational model, personas and user stories (three epics: Give feedback, Manage feedback, Share & collaborate); maintained as a living artefact ([Requirements.xlsx](#) in the repo root, mirrored as Jira stories [KAN-6–KAN-20](#)) where every requirement carries a user story, acceptance criteria, MoSCoW priority, source, and traceability to its implementing PR. The team contributed significantly to scoping: seven requirements go beyond the original brief (e.g. image-embedded project JSON, proposed by the team and client-approved in W4); the W3 scope change is traced to FR-03/10/11 and NFR-01. Of 18 requirements, 14 are **Done** (each with PR evidence), FR-15 partially delivered (QR sharing live, scan-to-import in S3), and 3 planned for S3/S4 (accessibility pass, Docker, user manual).

3.2 Frontend & Architectural Design

The UI follows a documented design language — warm, minimal editorial theme (warm paper [#f7f6f0](#), deep purple [#25213e](#), accent [#6b51c9](#)) — recorded in the repository guidelines and enforced in PR review. Key UX decisions came from evaluated alternatives: the feedback interface is a closable side panel (team vote) so reviewers never leave the canvas; clicking a comment highlights and navigates to its node; the editor is a dockable canvas workspace; image-only import restores a full project from a single dragged-in image.

Architecture is a **local-first SPA**: React 18 + TypeScript + Vite, maxGraph canvas engine, Redux Toolkit slices, React Router with an [adaptive basename](#) (same build runs on localhost and under the GitHub Pages sub-path), persistence via localStorage/idb-keyval, portability via image-embedded JSON (compressed, hardened against special characters). An optional SQLite/PostgreSQL path is preserved behind a documented abstraction for future growth at near-zero cost. Quality gates are part of the architecture: only green CI builds reach **develop**.

3.3 Coding, Testing, Deployment & Repository

- **Coding standards** are documented in the repository and enforced by automation plus review: ESLint zero-warning policy, Stylelint, conventional commit prefixes ([feat:/fix:/test:/chore:/doc:](#)), branch naming by work type, no direct commits to **develop/main** — every change lands via a [reviewed PR](#) (≥ 1 approval).
- **Testing**: QA-owned versioned test plan (v0.1 → v0.9 covering all merged user stories; v1.0 in S3). [20 Vitest unit/component test files](#) co-located with the code; [Cypress E2E specs](#) run in CI; every feature ships with tests in the same PR. All four reported bugs — logged as Jira tasks [KAN-21–KAN-23](#) and [KAN-37](#) — (feedback persistence [#17](#), project rename [#14](#), Do3 task bug [#18](#), node-edit/feedback loss on redraw [#23](#)) are fixed with regression tests.
- **Deployment**: any push to **develop** that passes CI is automatically built and published by the [CD job](#) to GitHub Pages — live at <https://lkevincc0.github.io/mm-local-editor/>. Three robustness fixes (PRs [#2](#), [#12](#), [#13](#)) make it work from anywhere; single-container Docker packaging is scheduled for S4 at the client's W6 request.
- **Repository**: documented GitFlow-lite model; [21 PRs](#) ([#2–#23](#)) merged during W3–W6, each traceable to a backlog item, bug or client request; hygiene actively maintained (stray submodule removed [#8](#), lockfiles current, `.env` template, CI submodule check added after the S1 retro).

4 Maintenance of Artefacts & Response to Feedback

All artefacts are living documents with dated, owned updates: requirements updated on every merged PR and on the W3 scope change; decision log (9 entries); contribution log reconciled weekly with [git history](#); bug log (all reported bugs closed with PR evidence); meeting minutes W3–W6; sprint plans/retros at every boundary; repo docs kept current as conventions evolved.

Feedback is logged, impact-assessed and actioned within the following sprint. Supervisor feedback (12 Aug: architecture finalised and agreed) was followed by detailed implementation through 19–26 Aug. Client feedback at the 27 Aug meeting (D3 feedback-system bug; make commenter-name editing, an “overall feedback” summary, and feedback image export more visible) was actioned in full — bug fixed, name-edit and overall-feedback features implemented with image export (PRs [#17–#20](#), [#23](#)).

Special circumstances handled: (i) the W3 client scope pivot removed all backend scope — Sprint 1 was re-planned the same week and the two affected members re-scoped to QA/Test Lead and DevOps, converting a scope loss into a quality investment; (ii) the Sprint-1 submodule incident broke fresh clones — root-caused and fixed the same week (PR [#8](#)), with a CI check added to prevent recurrence; (iii) five client-task follow-ups were re-allocated across all five members mid-sprint in W5 and all completed with PR evidence.

5 Plan for Weeks 7–12

Sprint	Goal	Key deliverables
S3 (W7–8)	Quality hardening	Test plan v1.0 (all user stories); CI lint gate + branch protection; coverage expansion; accessibility & responsive pass; QR scan-to-import
S4 (W9–10)	Deployment & polish	Single-container Docker image + compose; deployment guide; user manual + quick-start; UX polish from client notes
S5 (W11–12)	Final release	Feature freeze; full regression; client acceptance; v1.0 from <code>main</code> ; artefact audit; handover pack

Evidence index: [repository](#) · [merged PRs](#) · [commit history](#) · [CI/CD runs](#) · [CI/CD workflow](#) · [live deployment](#) · [Jira board](#) / [Jira issue list](#) (stories [KAN-6–KAN-20](#), bugs [KAN-21–KAN-23](#), sprint history [KAN-24–KAN-37](#)) · [Requirements.xlsx](#)